# Table of Contents

**Problem Statement**

Optimal Control for EDL (Entry , Descent and Landing) into Precise Martian Surface Location using SRP (Supersonic Retro Propulsion).

As we near our journey into the Martian Surface , we will have to execute one more powered guidance maneuver to land safely and precisely. This paper here is a review of all the different approaches available in literature to solve this problem as well as an attempt to come up with a relatively easy example of how one could solve this using Direct Methods and commonly available tools like MATLAB, Python etc.

The state variables (position, velocity and mass)

$$\boldsymbol{x(t)} = \begin{bmatrix} \boldsymbol{r} \\ \boldsymbol{v} \\ \boldsymbol{m} \end{bmatrix}$$

Equations of Motion :

$$\dot{\boldsymbol{r}}(\boldsymbol{t}) = \boldsymbol{v}(\boldsymbol{t}),$$

$$\dot{\boldsymbol{v}}(\boldsymbol{t}) = \boldsymbol{g} + \frac{\boldsymbol{T}(\boldsymbol{t})}{\boldsymbol{m}(\boldsymbol{t})},$$

$$\dot{\boldsymbol{m}}(\boldsymbol{t}) = -\boldsymbol{\eta}|\boldsymbol{T}(\boldsymbol{t})|$$

Thrust Constraints

$$\mathbf{0} < \boldsymbol{\rho_1} \leq |\boldsymbol{T}(\boldsymbol{t})| \leq \boldsymbol{\rho_2}$$

$$\boldsymbol{r}(\mathbf{0}) = \boldsymbol{r_0}, \quad \mathbf{v}(\mathbf{0}) = \mathbf{v_0}, \quad \boldsymbol{m}(\mathbf{0}) = \boldsymbol{m}_{\text{wet}} \ \textbf{(Initial Conditions)}$$

$$\boldsymbol{e_1^T r}(\boldsymbol{t_f}) = \mathbf{0}, [\boldsymbol{e_2}\ \boldsymbol{e_3}]^{\boldsymbol{T}} \boldsymbol{r}(\boldsymbol{t_f}) = \boldsymbol{q}\ , \dot{\boldsymbol{r}}(\boldsymbol{t_f}) = \mathbf{0}, \boldsymbol{m_f} \geq \boldsymbol{m_{dry}}$$

The objective function.

$$\boldsymbol{J} = -\boldsymbol{m}(\boldsymbol{t_f}) \textbf{ [Maximizing Final Mass, minimum fuel]}$$

## A : Necessary conditions of Optimality

Pontryagin's Maximum Principle is used to come up with the necessary conditions of Optimality.

**i) Hamiltonian Definition**

$$\boldsymbol{H(x(t),\lambda(t),T(t)) = 0 + \lambda_r^{\top}(t)v(t) + \lambda_v^{\top}(t)\left(g + \frac{T(t)}{m(t)}\right) - \alpha\lambda_m(t)(|T(t)|)}$$

$$\text{Where } |\boldsymbol{T(t)}| = \begin{Bmatrix} -T, T < 0 \\ +T, T > 0 \\ 0, T = 0 \end{Bmatrix}$$

$$\textbf{Subject to} \quad \boldsymbol{0 < \rho_1 \leq |T(t)| \leq \rho_2} \textbf{ (given)}$$

Where $\boldsymbol{\lambda_r, \lambda_v, \lambda_m}$ are the co-states associated with position, velocity and mass respectively.

**ii) Deriving the co-states equations.**

From the Euler Lagrange Equations , we get the Necessary conditions of optimality.

$\dot{\lambda_r}(t) = -\frac{\partial H}{\partial r} = 0$ which implies

$$\boldsymbol{\lambda_r(t) = K_1}$$

$\dot{\lambda_v}(t) = -\frac{\partial H}{\partial v} = -\lambda_r, which\ implies\ \lambda_v = -\lambda_r t + K_{2(\text{constant})}\ and\ finally$

$$\boldsymbol{\lambda_v(t) = -K_1 t + K_2}$$

$$\boldsymbol{\dot{\lambda_m}(t) = -\frac{\partial H}{\partial m} = \lambda_v(t)\frac{T(t)}{m(t)^2}}$$

**iii) Hamiltonian Minimization**

$$\boldsymbol{\frac{\partial H}{\partial T} = \frac{\lambda_v(t)}{m(t)} - \left(\alpha\lambda_m(t)\right)\frac{T(t)}{|T(t)|}} \qquad \textbf{where} \qquad \boldsymbol{\frac{\partial}{\partial T(t)}|T(t)| = \frac{T(t)}{|T(t)|}},$$

$$\textbf{Subject to } \boldsymbol{0 < \rho_1 \leq |T(t)| \leq \rho_2}$$

If this gradient changes sign along the optimal trajectory, then the control

T(t) should switch between its extreme values $\boldsymbol{\rho_1}$ and $\boldsymbol{\rho_2}$

1) **Fixed time problem**

The Hamiltonian and the co-state relations remain the same for the Fixed time problem as shown above.

The terminal cost $\phi(\mathrm{x}(\mathrm{t_f})) = -m(t_f)$, from the problem statement.

Boundary Conditions :

i) Initial States : Given $r(0) = r_0, \quad \mathrm{v}(0) = \mathrm{v}_0, \quad m(0) = m_{\text{wet}}$

ii) Terminal States : Given ,

$$e_1^T r(t_f) = 0, [e_2\ e_3]^T r(t_f) = q\ , \dot{r}(t_f) = 0, m_f \geq m_{dry}$$

iii) Co-state Boundary Conditions

$$\boldsymbol{\lambda(\mathrm{t_f})} = \boldsymbol{\partial}\frac{\phi(\mathrm{x}(\mathrm{t_f}))}{\boldsymbol{\partial}\mathrm{x}(\mathrm{t_f})}$$

$$\boldsymbol{\lambda_{\mathrm{r}(\mathrm{t_f})}} = \boldsymbol{\partial}\frac{-m(t_f)}{\boldsymbol{\partial}\mathrm{r}(\mathrm{t_f})} = 0$$

$$\boldsymbol{\lambda_{\mathrm{v}(\mathrm{t_f})}} = \boldsymbol{\partial}\frac{-m(t_f)}{\boldsymbol{\partial}\mathrm{v}(\mathrm{t_f})} = 0$$

$$\boldsymbol{\lambda_{\mathrm{m}(\mathrm{t_f})}} = \boldsymbol{\partial}\frac{-m(t_f)}{\boldsymbol{\partial}\mathrm{m}(\mathrm{t_f})} = -1$$

iv) $\frac{\boldsymbol{\partial H}}{\boldsymbol{\partial T}} = \frac{\boldsymbol{\lambda_v(t)}}{\boldsymbol{m(t)}} - \left(\alpha\boldsymbol{\lambda_m(t)}\right)\frac{\boldsymbol{T(t)}}{|\boldsymbol{T(t)}|} = \mathbf{0}$ will provide the optimal control T*(t) satisfied at all time t.

This problem can now be solved by integrating backwards in time.

2) **Free terminal time problem.**

If $\mathbf{t_f}$ is free , PMP states that the transversality condition should be

$$H(x(t_f), \lambda(t_f), T(t_f), t_f) + \boldsymbol{\partial}\frac{\phi(\mathrm{x}(\mathrm{t_f}), \mathrm{t_f})}{\boldsymbol{\partial}\mathrm{t_f}} = 0$$

Since , our terminal cost doesn't depend on the final time, the Hamiltonian at terminal time = 0

$$H(x(t_f), \lambda(t_f), T(t_f), t_f) = 0$$

$$H\left(x(t_f),\lambda(t_f),T(t_f)\right) = 0 + \lambda_r^{\top}(\mathrm{t_f})\mathrm{v}(\mathrm{t_f}) + \lambda_{\mathrm{v}}^{\top}(t_f)\left(\boldsymbol{g} + \frac{\boldsymbol{T(t_f)}}{m(t_f)}\right) - \alpha\lambda_m(t_f)\left(|\boldsymbol{T(t_f)}|\right)$$

The only term that remains is the one with the mass co-state

Therefore $H(t_f) = \alpha\left(|\boldsymbol{T(t_f)}|\right)$ , the first two terms are zero , because of the co-state being zero at terminal time.

$$H(t_f) = \alpha\left(|\boldsymbol{T(t_f)}|\right) = 0$$

Optimal thrusting scheme.

$$\frac{\partial \boldsymbol{H}}{\partial \boldsymbol{T}} = \frac{\boldsymbol{\lambda_v(t)}}{\boldsymbol{m(t)}} - \left(\alpha\boldsymbol{\lambda_m(t)}\right)\frac{\boldsymbol{T(t)}}{|\boldsymbol{T(t)}|} \qquad \textbf{where} \qquad \frac{\partial}{\partial \boldsymbol{T(t)}}|\boldsymbol{T(t)}| = \frac{\boldsymbol{T(t)}}{|\boldsymbol{T(t)}|}$$

This gives $$\frac{\boldsymbol{T(t)}}{|\boldsymbol{T(t)}|} = \frac{\frac{\boldsymbol{\lambda_v(t)}}{\boldsymbol{m(t)}}}{\left(\alpha\boldsymbol{\lambda_m(t)}\right)} = \frac{\boldsymbol{1}}{\boldsymbol{m(t)}\alpha}\frac{\boldsymbol{\lambda_v(t)}}{\boldsymbol{\lambda_m(t)}}$$

This relation doesn't explicitly provide the value of thrust instead it tells that the thrust vector could be aligned with the velocity co-state which also effects the mass co-states if we look at the evolution of co-states.

Since we cannot compute the intermediate Thrusts using the above relation and that the thrusts are bounded between two values (Min and Max) , and the final cost is only depended on the final mass and not thrust, we can conclude that a **bang-bang** control is the optimal control.

1. **Switching Condition:** If there's no range of T where $\frac{\partial H}{\partial T} = 0$ for a significant interval (i.e., no singular control), then T(t) must be at its bounds:

$$\frac{\partial \boldsymbol{H}}{\partial \boldsymbol{T}} = \frac{\boldsymbol{\lambda_v(t)}}{\boldsymbol{m(t)}} - \left(\alpha\boldsymbol{\lambda_m(t)}\right)\frac{\boldsymbol{T(t)}}{|\boldsymbol{T(t)}|}$$

If $\frac{\partial H}{\partial T} > 0$ , $i.e\ \frac{\lambda_v(t)}{m(t)} - \left(\alpha\lambda_m(t)\right)\frac{T(t)}{|T(t)|} > 0$ , Increasing T will increase the Hamiltonian, since we need to minimize , we decrease Thrust , or $\boldsymbol{\rho_1}$

$$\frac{\boldsymbol{\lambda_v(t)}}{\boldsymbol{m(t)}} > \left(\alpha\boldsymbol{\lambda_m(t)}\right)\frac{\boldsymbol{T(t)}}{|\boldsymbol{T(t)}|}$$

If $\frac{\partial H}{\partial T} < 0$ , $i.e\ \frac{\lambda_v(t)}{m(t)} - \left(\alpha\lambda_m(t)\right)\frac{T(t)}{|T(t)|} < 0$ , Increasing T will decrease the Hamiltonian, since we need to minimize , we increase Thrust , or $\boldsymbol{\rho_2}$

$$\frac{\boldsymbol{\lambda_v(t)}}{\boldsymbol{m(t)}} < \left(\alpha\boldsymbol{\lambda_m(t)}\right)\frac{\boldsymbol{T(t)}}{\boldsymbol{|T(t)|}}$$

**The evolution of the co-state dynamics will give a better switching insight.**

## B. Simulation of the free terminal time problem

The Minimum-fuel landing trajectory for the data provided was calculated using direct method in Matlab, particularly parameter optimization approach using trapezoidal collocation and the Fmincon function in Matlab.

In the MATLAB solution I used an alternative objective function

$$\boldsymbol{J = \int_0^{t_f} |T(t)|dt}$$ **which is analogous to J = -m($t_f$)**

**Time grid : Chose N discretization points**

$$\Delta t = \frac{t_f}{N-1}$$

**Parameterization of states and control :**

$$x_i \approx x(t_i), u_i \approx u(t_i), i = 0, \dots, N.$$

**Trapezoidal Rule for State Dynamics**

$$x_{i+1} = x_i + \frac{\Delta t}{2}[f(x_i, u_i) + f(x_{i+1}, u_{i+1})], i = 0, \dots, N-1$$

**Trapezoidal Rule for Control**

$$T_i \approx T(t_i), i = 0, \dots, N.$$

$$\| T_i \| = \sqrt{T_{i,1}^2 + T_{i,2}^2 + T_{i,3}^2 \dots}$$

Where 1, 2 and 3 are there thrust vectors directions.

**Trapezoidal Rule to Approximate the Integral:**

$$J_i = \sum_{i=0}^{i=1} \frac{\Delta t}{2}(\| T_i \| + \| T_{i+1} \|)$$

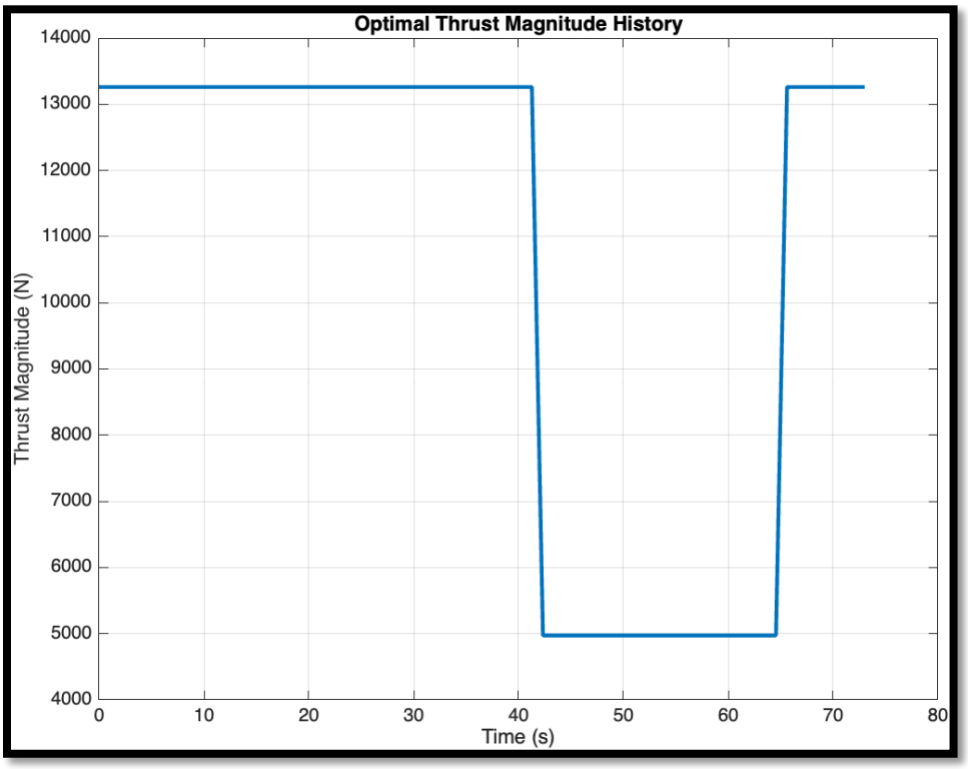


Figure 1: Optimal Thrust Profile without altitude constraint

**`Optimal Terminal Time (s): 73.02 ; Remaining Fuel (kg): 48.85`**

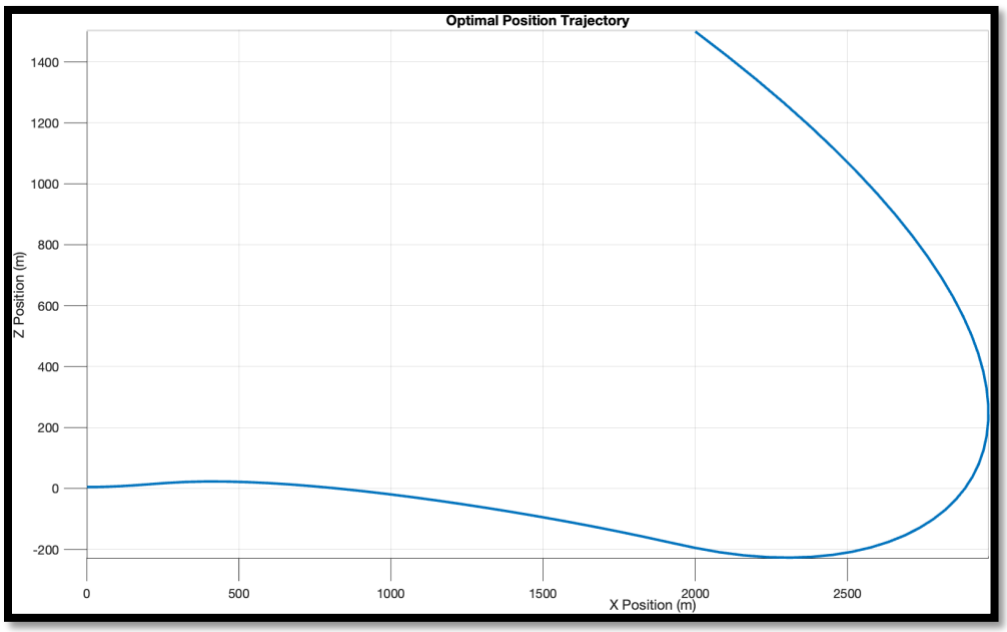


Figure 2:Front view of the trajectory without  altitude constraint

You can see that it goes below 0.

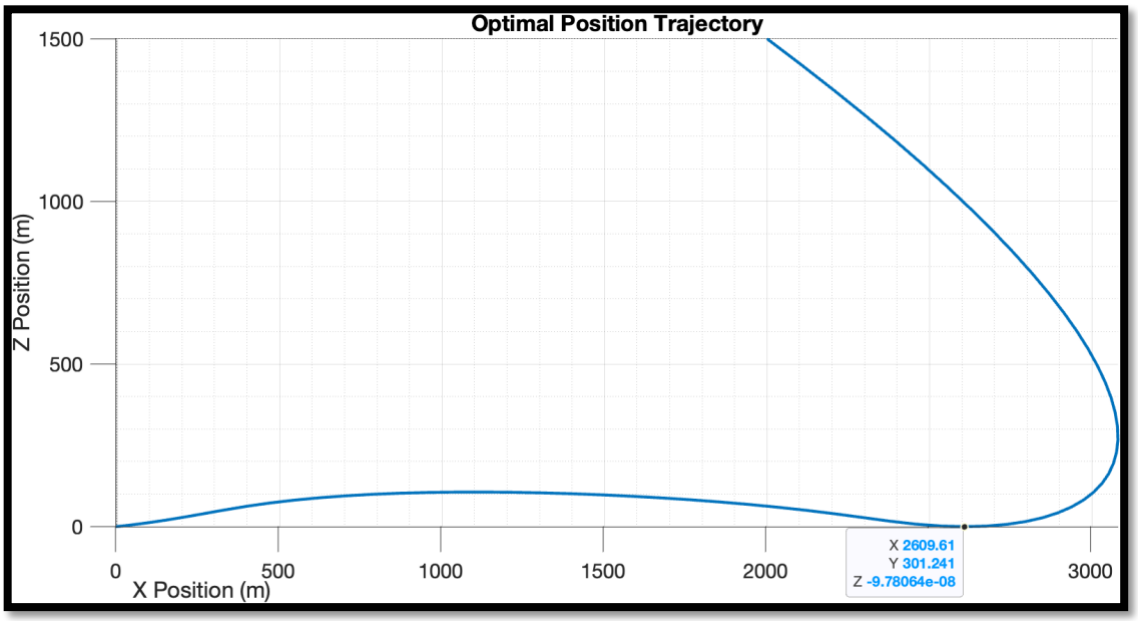


Figure 3:Front view of the trajectory with altitude constraint

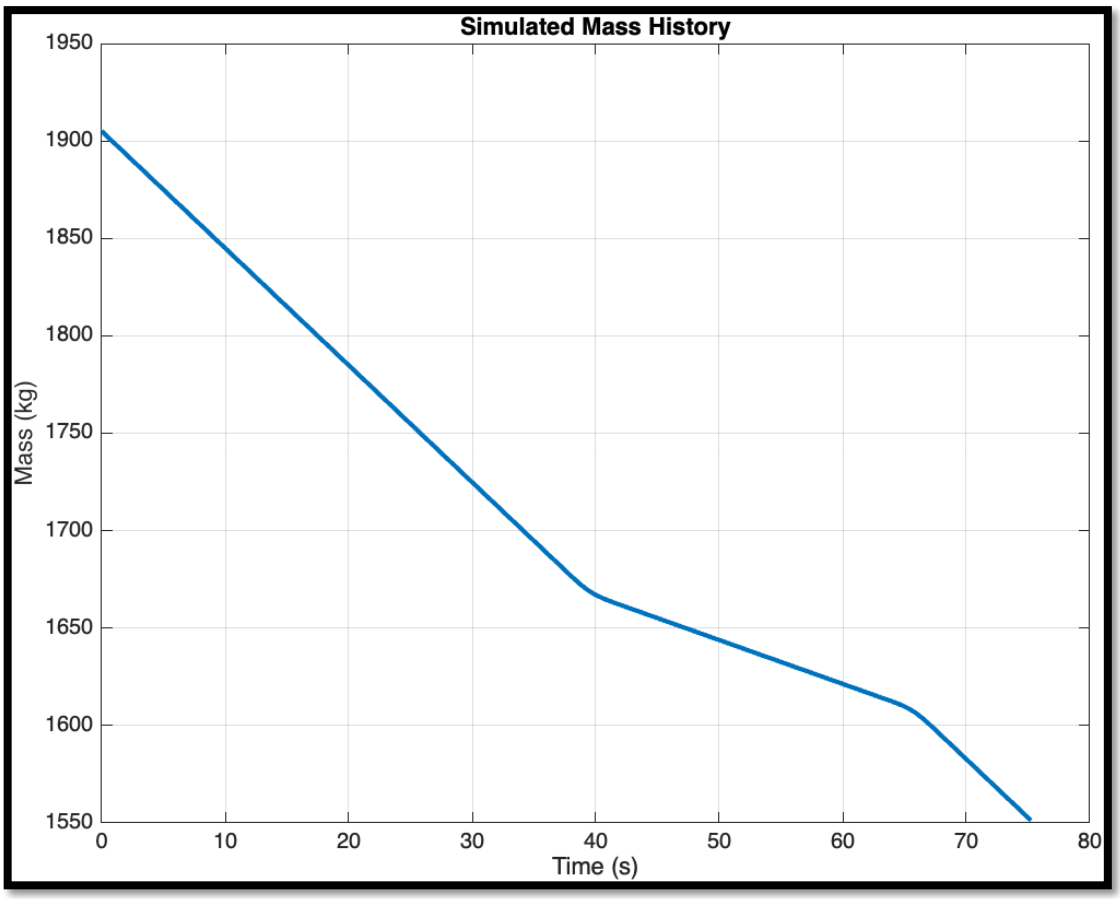


Figure 4 :Optimal Mass History with altitude constraint

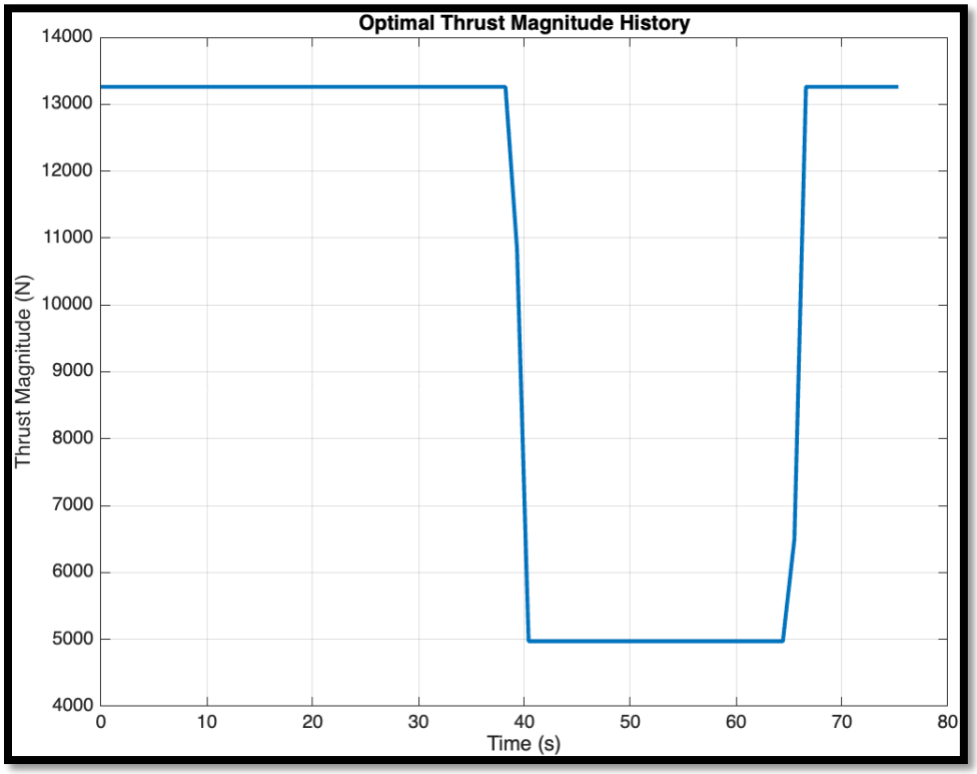


Figure 5 :Optimal Thrust Profile with altitude constraint

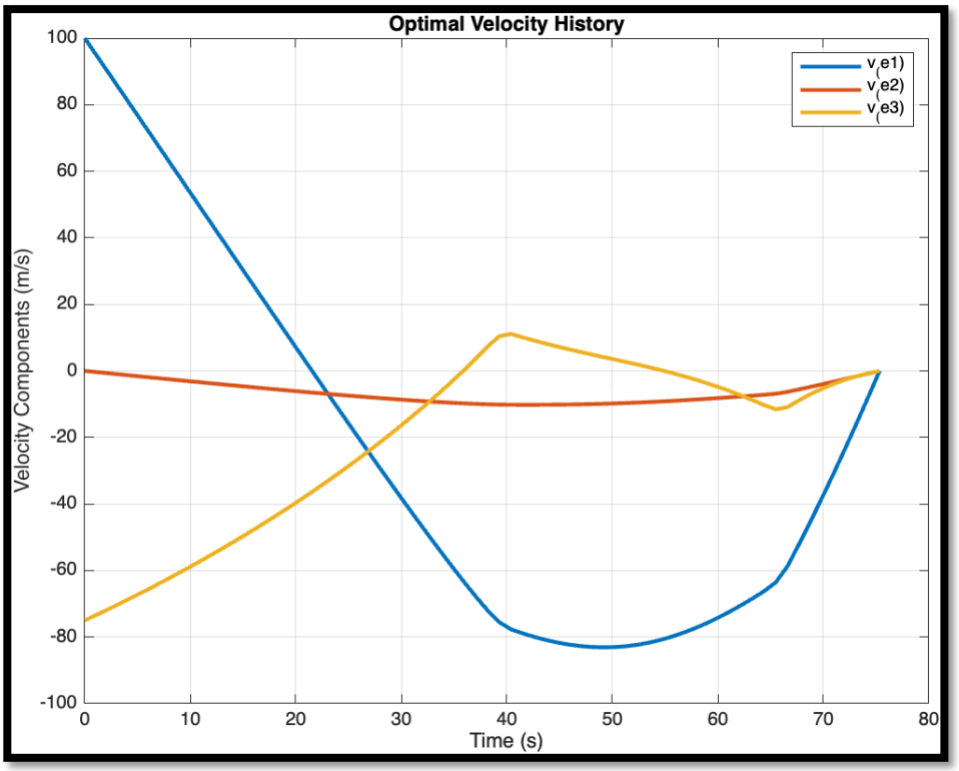


Figure 6: Optimal Velocity Profile with altitude constraint

## Results

Optimal Terminal Time (s): 75.33

**Remaining Fuel (kg): 46.35**

I tried both the objective functions; they provided similar results, but I got smoother results with the Thrust in the objective function.

Observation : I had to constrain the altitude to not be less than zero , else the trajectory goes into the ground. See figure above.

The initial condition and the value of N played an important role, I started with 70% max thrust on the $-e_1$ direction and a N of 70 , SQP with about 450 iterations with Parallel mode on.

## C. Glide slope Constraint

To introduce a glide-slope constraint given such that

$$\left\|S[r(t) - r(t_f)]\right\| - c^T\left[r(t) - r(t_f)\right] \leq 0$$

Where $S = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$ , $c = \frac{e_1}{tan\Upsilon}$ $\Upsilon$ and $\Upsilon = 4^o$

This problem adds a new constraint to the position vector that must be accounted for during the entirety of the trajectory.

### i) Changes in Part A

**State dynamics remain unchanged.**

We introduce a modified Hamiltonian $\widetilde{H}$ where ,

$$\boldsymbol{\widetilde{H} = H + \mu(t)y(r(t))}$$

Here H is the original Hamiltonian , y(r(t)) is the glide slope constraint provided above and $\mu$ is the Lagrange multipliers associated to augment the glide slope constraint y (r(t)).

$$\boldsymbol{\widetilde{H}\left(x(t), \lambda(t), T(t), \mu(t)\right) = \lambda_r^\top(t)v(t) + \lambda_v^\top(t)\left(g + \frac{T(t)}{m(t)}\right) - \alpha\lambda_m(t)(|T(t)|) + \mu\{\left\|S\left[r(t) - r(t_f)\right]\right\| - c^T\left[r(t) - r(t_f)\right]\}}$$

The co-state dynamics changes accordingly ,

$\dot{\lambda_r}(t) = -\frac{\partial H}{\partial r} = -\mu\left\{S\frac{S([r(t)-r(t_f)])}{\|S[r(t)-r(t_f)\|} - c\right\}$, **the only co-state equation that changes.**

$\dot{\lambda_v}(t) = -\frac{\partial H}{\partial v} = -\lambda_r$ no change since no velocity terms in the glide slope constraint.

$$\dot{\lambda_m}(t) = -\frac{\partial H}{\partial m} = \lambda_v(t)\frac{T(t)}{m(t)^2}$$

The optimality constraint remains the same because the new constraint has no Thrust terms in it.

$$\frac{\partial H}{\partial T} = \frac{\partial \tilde{H}}{\partial T} = \frac{\lambda_v(t)}{m(t)} - \left(\alpha \lambda_m(t)\right) \frac{T(t)}{|T(t)|}$$

Since there are no Thrust implications from the new constraint, there is no change in the Thrust scheme , it still is a bang-bang control.

The boundary conditions for costate still apply the same ,

Since $J = -m\left(t_f, x(t_f)\right)$ ,

The costate at final time remain same,

$$\lambda_{r(t_f)} = \partial \frac{-m(t_f)}{\partial r(t_f)} = 0$$

$$\lambda_{v(t_f)} = \partial \frac{-m(t_f)}{\partial v(t_f)} = 0$$

$$\lambda_{m(t_f)} = \partial \frac{-m(t_f)}{\partial m(t_f)} = -1$$

**The problem has provided $h \leq 0$**

Conclusion : The introduction of $\mu(t)$ and the modified co-state equation for position $\lambda_r$ are the key differences in defining the condition of optimality.

### ii) Changes in Part B

There will be an additional inequality constraint added to the problem for the glide slope which is implemented as follows.

$\|S[r(t) - r(t_f)]\|$ can be written as $\sqrt{\|S[r(t) - r(t_f)]\|^2}$ or

$$\sqrt{(S(r(t) - r(t_f))^T (S\left(r(t) - r(t_f)\right))}$$

The discretization of the constraint becomes

$$\boldsymbol{y(r_i) = \sqrt{(S(r_i - r_f)^T (S(r_i - r_f))} - c^T(r_i - r_f) \leq 0}$$

I tried to simulate this in Matlab using Fmincom and different methods (sqp, interior points etc. ) but couldn't solve it.

I tried using ipopt solver within the casadi package in python , I couldn't converge to a solution that way either.

# I. Matlab Code

```
clear; clc; close all;


% Martian Gravity (m/s^2)
g = [-3.711400; 0; 0];


% Simulation Data
m_wet = 1905;  % Initial mass(kg)
m_dry = 1505;  %(kg)
alpha = 4.53e-4;% Fuel constant (s/m)
rho_1 = 4972;   % Minimum thrust
rho_2 = 13260;  % Maximum thrust
r0 = [1500; 500; 2000];     % Position in meters
v0 = [-75; 0; 100];         % Velocity in m/s
rf = [0; 0; 0]; % Terminal position
vf = [0; 0; 0]; % Terminal velocity


% Final Time guess
tf_guess = 200;   % Terminal time guess
N = 70;           % Discretization point


t = linspace(0, tf_guess, N)'% Time variable
dt = (t(end) - t(1)) / (N - 1);% Time step)
r = zeros(N, 3);   %position states
v = zeros(N, 3);   %velocity states
m = zeros(N, 1);   %Mass states
T = zeros(N, 3);   % thrust vector (T)
```

```
% Initial guesses for states and controls
for i = 1:N
    tau = (i - 1) / (N - 1);
    r(i, :) = r0' + tau * (rf' - r0');
    v(i, :) = v0' + tau * (vf' - v0');
    m(i) = m_wet - tau * (m_wet - m_dry);
    T(i, :) = [0.7*rho_2, 0, -2000];  % This is the best I could come up , any deviation and its goes off.
end
% One big initial variables single vector
X0 = [reshape(r, [], 1); reshape(v, [], 1); m; reshape(T, [], 1); tf_guess];


% Bounds on states and controls
lb_r = -10000 * ones(N * 3, 1);
ub_r = 10000 * ones(N * 3, 1);
lb_v = -1000 * ones(N * 3, 1);
ub_v = 1000 * ones(N * 3, 1);
lb_m = m_dry * ones(N, 1);
ub_m = m_wet * ones(N, 1);
lb_T = -rho_2 * ones(N * 3, 1); % this is the only way it converged
ub_T = rho_2 * ones(N * 3, 1);
% Bounds on terminal time
lb_tf = 30;
ub_tf = 200;
% Combine bounds
lb = [lb_r; lb_v; lb_m; lb_T; lb_tf];
ub = [ub_r; ub_v; ub_m; ub_T; ub_tf];


options = optimoptions('fmincon', 'Algorithm', 'sqp','UseParallel',true,'Display', 'iter-detailed', ...
                       'MaxFunctionEvaluations', 2e6, 'MaxIterations', 2000, ...
                      'ConstraintTolerance', 1e-3, 'Diagnostics', 'on');
```

```
%fmincon optimizer
[X_opt, ~, exitflag, output] = fmincon(@objective, X0, [], [], [], [], lb, ub, @constraints, options);


%optimization verification
if exitflag <= 0
    error('No Convergence');
end


% Extract variables from optimal solution
N_opt = (length(X_opt) - 1) / 10;
tf_opt = X_opt(end);
dt_opt = tf_opt / (N_opt - 1);
t_opt = linspace(0, tf_opt, N_opt)';
r_opt = reshape(X_opt(1:N_opt*3), N_opt, 3);
v_opt = reshape(X_opt(N_opt*3+1:2*N_opt*3), N_opt, 3);
m_opt = X_opt(2*N_opt*3+1:2*N_opt*3+N_opt);
T_opt = reshape(X_opt(2*N_opt*3+N_opt+1:2*N_opt*3+N_opt+N_opt*3), N_opt, 3);
% Compute thrust magnitudes
T_mag_opt = sqrt(sum(T_opt.^2, 2));


% Thrust Plot
figure;
plot(t_opt, T_opt(:,1), t_opt, T_opt(:,2), t_opt, T_opt(:,3), 'LineWidth', 2);
grid on;
xlabel('Time (s)');
ylabel('Thrust Components (N)');
legend('T_(e1)', 'T_(e2)', 'T_(e3)');
title('Optimal Thrust History');
```

```
% Thrust Magnitude Plot
figure;
plot(t_opt, T_mag_opt, 'LineWidth', 2);
grid on;
xlabel('Time (s)');
ylabel('Thrust Magnitude (N)');
title('Optimal Thrust Magnitude History');


%% Display Optimal Terminal Time and Final Mass
fprintf('Optimal Terminal Time (s): %.2f\n', tf_opt);
final_mass = m_opt(end);
fprintf('Final Mass of the Lander (kg): %.2f\n', final_mass);
initial_fuel = m_wet - m_dry;
final_fuel = final_mass - m_dry;
fprintf('Remaining Fuel (kg): %.2f\n', (remaining_fuel);


% Use the optimized thrust profile for simulation
thrust_profile = @(t) interp1(t_opt, T_opt, t, 'linear', 'extrap'); %learnt this from a matlab tutorial.
% ODE Function for Dynamics
function dXdt = dynamics(t, X, g, alpha, thrust_profile)
    % States: position (r), velocity (v), mass (m)
    r = X(1:3);
    v = X(4:6);
    m = X(7);
    g = g(:);
    % Thrust from Thrust profile
    T = thrust_profile(t);
    T = T(:);
    % Dynamics
```

```
    drdt = v;
    dvdt = g + T / m;
    dmdt = -alpha * norm(T);
    dXdt = [drdt; dvdt; dmdt];% Derivatives
end


X0 = [r0; v0; m_wet];% Initial state vector
tspan = [0, tf_opt];% Simulation Time
% Solve the ODEs using ode45
[t_sim, X_sim] = ode45(@(t, X) dynamics(t, X, g, alpha, thrust_profile), tspan, X0);
% Extract position, velocity, and mass from simulation
r_sim = X_sim(:, 1:3);
v_sim = X_sim(:, 4:6);
m_sim = X_sim(:, 7);


% Position Plot
figure;
plot3(r_opt(:,3), r_opt(:,2), r_opt(:,1), 'LineWidth', 2);
grid on;
xlabel('X Position (m)');
ylabel('Y Position (m)');
zlabel('Z Position (m)');
title('Optimal Position Trajectory');
axis equal;


% Velocity Plot
figure;
plot(t_opt, v_opt(:,3), t_opt, v_opt(:,2), t_opt, v_opt(:,1), 'LineWidth', 2);
grid on;
xlabel('Time (s)');
```

```
ylabel('Velocity Components (m/s)');
legend('v_(e1)', 'v_(e2)', 'v_(e3)');
title('Optimal Velocity History');


% Mass Plot from Simulation
figure;
plot(t_sim, m_sim, 'LineWidth', 2);
grid on;
xlabel('Time (s)');
ylabel('Mass (kg)');
title('Simulated Mass History');


%% Objective Function
function J = objective(X)
    % % Extract variables
    N = (length(X) - 1) / 10;
    tf = X(end);
    dt = tf / (N - 1);
    T = reshape(X(7*N+1:10*N), N, 3);
    T_mag = sqrt(sum(T.^2, 2));  % Compute thrust magnitudes

    % Compute the objective (approximate integral using trapezoidal rule)
    J = dt * trapz(T_mag);
    %J =  dt * (sum(T_mag) - 0.5 * (T_mag(1) + T_mag(end))); Tried
    %different ways to do this. the trapz function seems to work well
    % to use -m(t_f) as the objective function.
    % m = X(2*N*3+1:2*N*3+N); This is to
    % J = - m(end);
end
```

```
%% Constraints Function
function [c, ceq] = constraints(X)
    N = (length(X) - 1) / 10;  % Total number of time points
    tf = X(end);
    dt = tf / (N - 1);
    % Given Data
    g = [-3.7114; 0; 0];
    alpha = 4.53e-4;
    m_wet = 1905;
    m_dry = 1505;
    rho_1 = 4972;
    rho_2 = 13260;
    % States
    r = reshape(X(1:N*3), N, 3);
    v = reshape(X(N*3+1:2*N*3), N, 3);
    m = X(2*N*3+1:2*N*3+N);
    % Controls
    T = reshape(X(2*N*3+N+1:2*N*3+N+N*3), N, 3);
    c = []; %inequality constraints
    ceq = [];%equality constraints
    % Dynamics constraints (collocation)
    for i = 1:N-1
        dt_i = dt;  % Time step
        % States at current and next time steps
        r_i = r(i, :)';
        r_ip1 = r(i+1, :)';
        v_i = v(i, :)';
        v_ip1 = v(i+1, :)';
        m_i = m(i);
        m_ip1 = m(i+1);
        % Controls at current and next time steps
```

```
    T_i = T(i, :)';
    T_ip1 = T(i+1, :)';
    % Average values
    v_avg = 0.5 * (v_i + v_ip1);
    m_avg = 0.5 * (m_i + m_ip1);
    T_avg = 0.5 * (T_i + T_ip1);
    % Dynamics equations using trapezoidal rule
    dr = r_ip1 - r_i - dt_i * v_avg;
    dv = v_ip1 - v_i - dt_i * (g + T_avg / m_avg);
    dm = m_ip1 - m_i + dt_i * alpha * sqrt(sum(T_avg.^2));

    % equality constraints
    ceq = [ceq; dr; dv; dm];
end
% Initial conditions equality
ceq = [ceq;
       r(1, :)' - [1500; 500; 2000];
       v(1, :)' - [-75; 0; 100];
       m(1) - 1905];
ceq = [ceq; % Terminal conditions equality
       r(end, :)' - [0; 0; 0];
       v(end, :)' - [0; 0; 0]];


c = [c; % Mass constraint at final time
     1505 - m(end)];
c = [c; -r(:,1)]; % Altitude constrained added
%Thrust magnitude constraints (inequalities)
for i = 1:N
    T_i = T(i, :)';
    T_mag = norm(T_i);
    c = [c;
```

```
            rho_1 - T_mag;
            T_mag - rho_2];
    end
end
```